
Amazon Review Predictions

Colin Cannell, Tim Golombeck, Carter Howell

— Rochester Institute of Technology —

CSCI-335

April 22, 2026

Intro

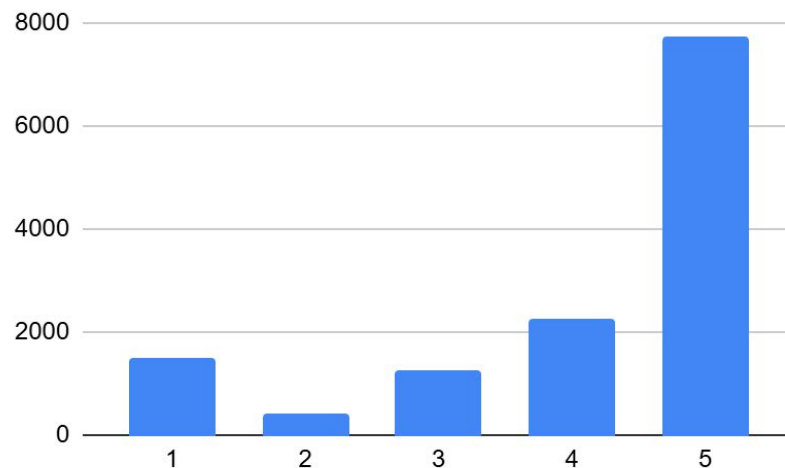
- Problem: Online reviews are helpful for consumers to make decisions on what products to buy, but star ratings can be inconsistent with the review's text and be misleading
- Goal: Accurately predict a star rating (1-5) from the text of a review

Hypothesis

- Models used for this approach:
 - Linear Regression
 - Logistic Regression
 - Decision Trees
 - Random Forest
 - Transformer
- Prediction: Linear Regression will perform the worst since this is a multi-class classification problem, while the Transformer model performs the best due to its complexity

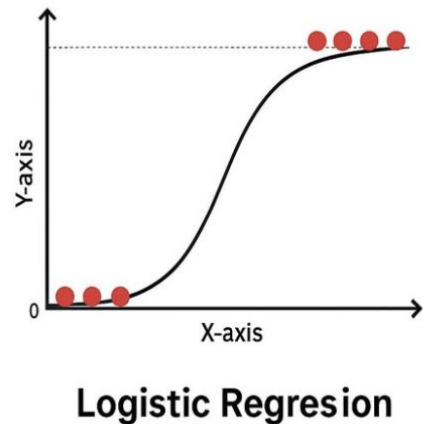
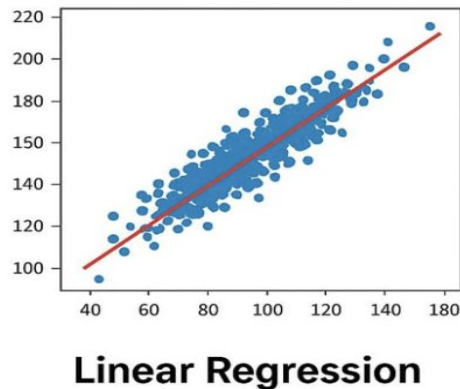
Dataset

- Over 13,000 samples of data
- Each piece of data contains:
 - Text review
 - Integer 1-5 indicating a star rating
- Data is imbalanced towards 5 star ratings



Preprocessing: Linear & Logistic Regression Models

- Text converted into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF)
- Models Used:
 - Linear Regression (Baseline)
 - Logistic Regression (Classification)
- Train/Test Split: 80%/20%

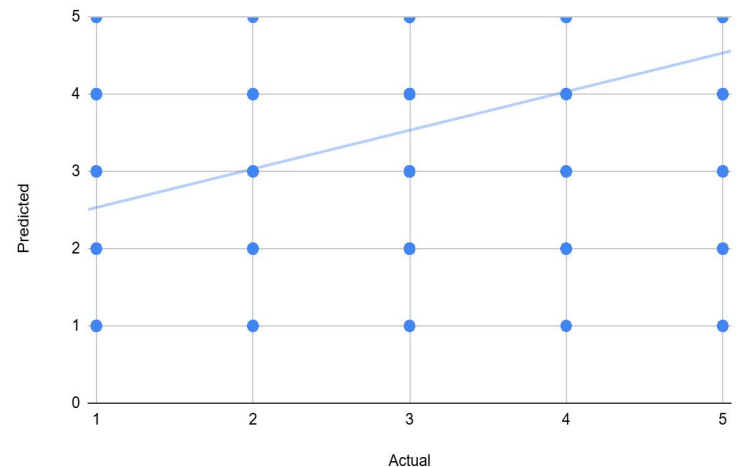
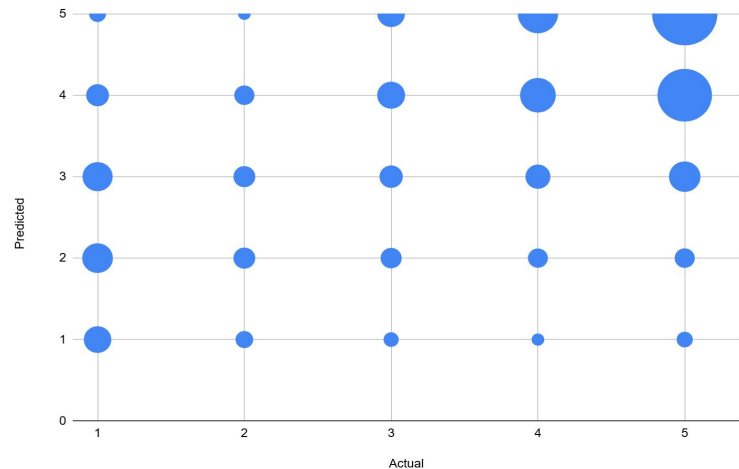


Methods: Linear Regression Model

- Linear Regression is used to predict continuous values
- Trained using batch gradient descent
- Output is rounded to the nearest rating (1-5 stars)

Results: Linear Regression Model

- Training Accuracy: 50.46%
- Testing Accuracy: 44.34%
- Training RSS: 7702
- Testing RSS: 3253
- High RSS and low accuracy indicates that the model is underfitting



Methods: Logistic Regression Model

- Logistic Regression is used for classification tasks such as this
- Predicts probability of each star rating class

Results: Logistic Regression Model

- Training Accuracy: 67.03%

- Testing Accuracy: 65.77%

	Prediction: 1	Prediction: 2	Prediction: 3	Prediction: 4	Prediction: 5
Actual: 1	245	1	5	7	64
Actual: 2	47	1	3	12	27
Actual: 3	31	0	5	13	168
Actual: 4	16	0	6	26	446
Actual: 5	25	1	2	57	1512

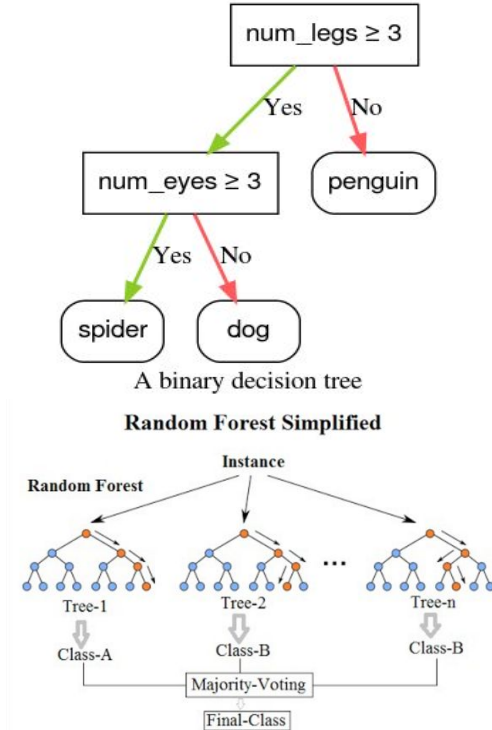
- Cross Validation Scores: [60.85%, 64.26%, 61.18%, 61.09%, 56.60%]

- Average Cross Validation: 60.80%

- CV Scores being a bit lower suggests the model could have slight overfitting or variance

Preprocessing: Decision Tree and Random Forest Models

- Started by Tokenizing Text of Every Review
- Feature Engineering: Binary Bag-Of-Words, 1 if Word in Vocab Appears in Review, 0 Otherwise.
- For simplicity my BoW vectors for each review only include the words that appear in both that review and was in my vocabulary.
- Train/Test Split 80%/20%



Methods: Decision Tree and Random Forest Models

- Used Weighted Gini/Gini Gain and Weighted Entropy/Info Gain to determine Best Attribute to Split on.
- To handle class imbalance in Amazon ratings, applied class weights
- Then Generated Decision Tree for each Hyperparameter to determine best HyperParameters for Decision Tree and Random Forest (342 Models Created)

Results: Decision Tree Model

- Model Performance Based on Standard Multi-Class Accuracy and Binary Accuracy (Generalized)
- Binary Accuracy was determined by converting the rating problem into a two-class classification task: reviews with a 5-star rating were labeled as class 1, and all other ratings (1–4 stars) were labeled as class 0.
- Used the Binary Accuracy to find TPR and FPR to generate an ROC curve.

Results: Decision Tree Model

===== RESULTS SUMMARY =====

Best Accuracy Model: dt_179_En_MD5_MSS2_MSL5_MIG0.0001_MF500_42.json

Best Accuracy: 0.5618

Best Generalized Model: dt_305_En_MD15_MSS5_MSL5_MIG0.0001_MF500_42.json

Best Generalized Accuracy: 0.6438

Best ROC Model: dt_305_En_MD15_MSS5_MSL5_MIG0.0001_MF500_42.json

Best ROC Score (TPR - FPR): 0.2469

ROC AUC Score: 0.6275

Top 5 Models by Generalized Accuracy:

	model	generalized_accuracy	tpr	fpr
304	dt_285_En_MD15_MSS2_MSL5_MIG0_MFNone_42.json	0.64375	0.787958	0.541107
14	dt_306_En_MD15_MSS5_MSL5_MIG0.0001_MFNone_42.json	0.64375	0.787958	0.541107
267	dt_286_En_MD15_MSS2_MSL5_MIG0.0001_MF200_42.json	0.64375	0.787958	0.541107
266	dt_288_En_MD15_MSS2_MSL5_MIG0.0001_MFNone_42.json	0.64375	0.787958	0.541107
32	dt_319_En_MD15_MSS10_MSL5_MIG0_MF200_42.json	0.64375	0.787958	0.541107

Total Models Evaluated: 324

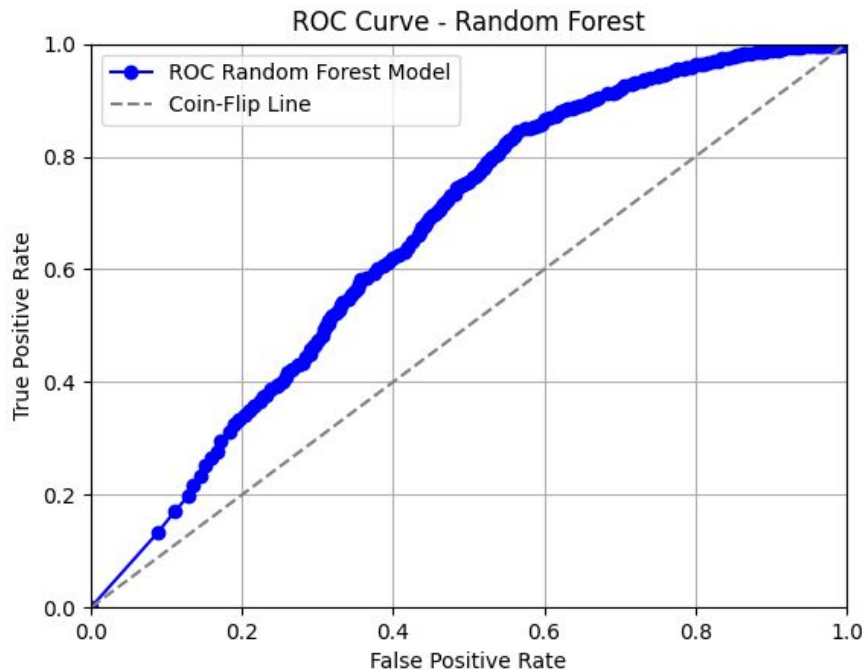
=====

Methods: Random Forest Model

- Use Best HyperParameters from Decision Tree
- Use Bootstrapping on Training Data to create Bootstrapped Samples for use in Random Forest
- Generate 200 Decision Trees to add into the Random Forest and select rating based on the majority prediction.

Results: Random Forest Model

- Built the ROC curve by changing the thresholds on the Random Forest's vote score and measuring how TPR and FPR change.

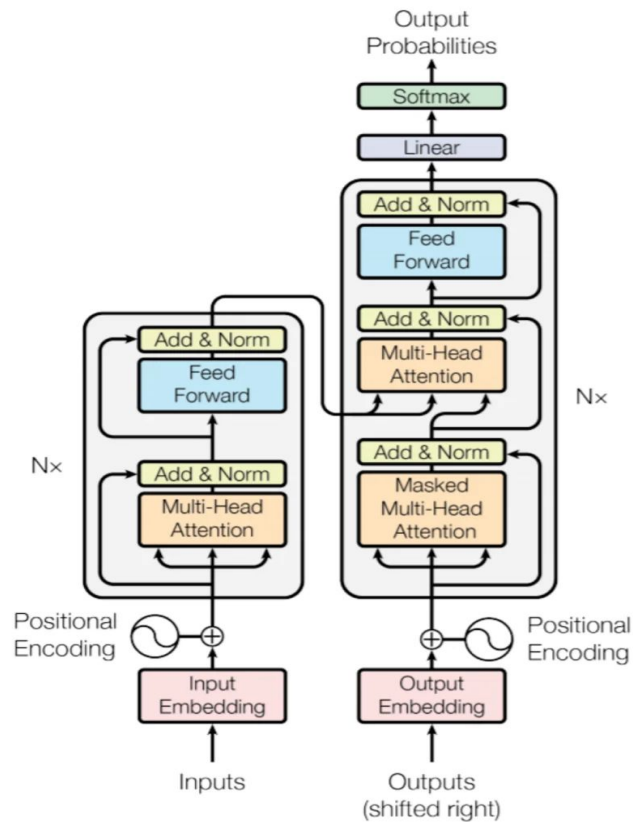


Results: Random Forest Model

```
===== RESULTS =====  
Accuracy:          0.5930  
Binary Accuracy:  0.6301  
TPR (last):       0.1335  
FPR (last):       0.0889  
AUC:              0.6534  
=====
```


Transformer Model

Transformers are a form of deep learning models which process its input as an entire sequence in parallel, this allows them to capture long range relationships between words



Preprocessing: Transformer Model

Preprocessing: dropped NaNs, stripped emojis/non-alphanumeric characters with a regex, and rescaled ratings from 1–5 to 0–4.

Tokenization: used Hugging Face's GPT-2 AutoTokenizer (BPE) with padding and truncation to 512 tokens. The EOS token doubled as the pad token.

ReviewDataset: a custom `torch.utils.data.Dataset` that wraps the tokenized encodings and copies `input_ids` into labels so the Transformer can train as a language model

Methods: Model Construction



Transformer was implemented from scratch in PyTorch
each sub-module is built as its own `nn.Module`:

- **scaled_dot_product_attention** — the raw attention formula, with optional masking.
- **MultiHeadAttention** — projects inputs, splits them into 8 heads, applies attention, then concatenates and projects back.
- **PositionalEncoding** — precomputed sine/cosine table registered as a buffer so it moves with the model to GPU.
- **EncoderLayer** — self-attention → residual + LayerNorm → feed-forward → residual + LayerNorm.
- **DecoderLayer** — adds a second *cross-attention* sub-layer so the decoder can attend to encoder outputs.
- **Transformer** — wires it all together with source/target embeddings, 6 encoder + 6 decoder layers, and a final linear head to vocab size.

Results: Transformer Model



Conclusion: Regression vs Classification

- Linear Regression underfits the data and isn't suitable for this task
- Logistic Regression might slightly overfit, but it's better designed for the problem and has higher accuracy
- Takeaways: Our prediction of linear regression having the worst performance was correct. Classification models outperform regression for rating predictions.

Conclusion: Decision Tree vs Random Forest

- Random Forest slightly outperformed Decision Trees in overall accuracy.
- Both Models performed better on generalized (binary) data.
- Both beat the Linear model in terms of accuracy, but also both fell short of Transformer and Logistic Regression.
- **Overall, neither model was the top performer, suggesting that more advanced or better-tuned approaches are needed for this dataset.**

Conclusion: Transformer Model

The Transformer model's high accuracy in predicting Amazon review sentiment is not surprising given its architectural complexity

Multi-head attention enables the model to analyze multiple aspects of a review simultaneously (e.g., tone, keywords, nuance)

The added complexity of Transformers directly translates into deeper language understanding, making them highly effective for real-world NLP tasks like sentiment analysis

Questions?

Acknowledgements

<https://www.kaggle.com/datasets/abhayparashar31/amazon-reviews-rating-prediction>

<https://www.kaggle.com/datasets/lokeshbaviskar/amazon-rating-prediction-based-on-reviews>